

CausalDrive

Scalable multimodal world models for embodied intelligence

Core objective

Develop efficient world-model pretraining across vision, action and state; use autonomous driving as the primary testbed, then validate transfer to a second embodied environment. Deliver research evidence and a production-grade Python/C++/GPU system.

PROJECT LEAD

Daniel Davis

PLANNED DURATION

18–24 months, part-time

PRIMARY LANGUAGES

Python · C++20

TARGET OUTCOME

First-author research + scalable ML system

Multimodal data

Pretraining

World rollout

Policy / task

GPU deployment

CausalDrive is the first concrete application, not the permanent boundary. The programme develops transferable expertise in multimodal/video foundation models, controllable generation, distributed training and accelerated inference for driving, robotics and frontier research engineering.

Research question and system features

Research question: How can multimodal world models learn efficient, action-controllable and uncertainty-aware representations that support real-time decisions—and which principles transfer between autonomous driving and other embodied agents?

1 Multimodal pretraining

Jointly model video, agent actions, physical state and optional language/task context using self-supervised objectives.

2 Structured latent state

A compact visual representation augmented with ego motion, drivable-area, lane and dynamic-object features.

3 Action-conditioned rollout

A small recurrent or transformer dynamics model predicts 0.5–2 seconds ahead under a proposed action sequence.

4 Multiple futures

Probabilistic or ensemble prediction represents ambiguity rather than collapsing every situation into one average future.

5 Action fidelity

Counterfactual tests verify that steering and speed actually change predictions in physically and geometrically consistent ways.

6 Uncertainty

Calibration and out-of-distribution tests identify when rollout predictions should not be trusted by a planner.

7 Scalable ML systems

Multi-GPU training, data streaming, mixed precision and profiling connect local research to foundation-model engineering.

8 Deployment and transfer

C++/CUDA inference supports CARLA closed loop, followed by one small navigation or manipulation transfer study.

Explicit non-goals

The project will not attempt frontier-scale pretraining, an entire driving stack, or several full robotics projects. Scaling is studied through controlled experiments; a second domain remains a small transfer test rather than a competing project.

Primary hypothesis

Action-consistency objectives, planning-aware latents and dynamic token routing can improve controllability and task utility per unit of compute; the resulting principles should transfer beyond the original driving domain.

Experiments and potential contributions

A. Action-fidelity benchmark

Hold the initial scene fixed while varying steering and speed. Compare recorded, zeroed, reversed and shuffled actions. Measure whether predicted ego motion, lanes, objects and optical flow respond correctly.

Most feasible paper Low-medium compute

B. Planning-aware representation

Compare reconstruction-oriented latents against structured states trained for ego motion, occupancy and risk. Test whether visual quality predicts downstream planning performance.

Strong research question Medium compute

C. Calibrated multiple futures

Compare deterministic, ensemble and probabilistic rollouts. Evaluate calibration, diversity, risk coverage and behaviour under weather, lighting and action distribution shifts.

Research extension Medium compute

D. Scalable, action-aware efficiency

Allocate tokens by action and horizon; measure quality, throughput and memory from one GPU to multi-GPU training, then validate the method in a second embodied task.

Links to Lava-Prune Novelty candidate

Required baselines

Baseline	Purpose	Key comparison
Repeat last state/frame	Measures the value of learning any dynamics.	Short-horizon latent and perceptual error
Constant velocity / optical flow	Tests whether the model beats simple motion assumptions.	Geometry and ego-motion consistency
Linear or MLP dynamics	Controls for model complexity.	Accuracy, parameters and latency
Action-agnostic architecture	Determines whether action conditioning adds information.	Counterfactual fidelity and rollout error
Shuffled/zero actions	Detects models that ignore control input.	Action sensitivity and physical plausibility

Evaluation groups

Prediction Latent cosine/MSE, trajectory error, heading, optional LPIPS/SSIM.	Control fidelity Response direction, magnitude, monotonicity and scene consistency.	Planning Collision proxy, lane deviation, progress, comfort and success rate.
Uncertainty Calibration error, NLL, risk-coverage and OOD detection.	Systems p50/p95/p99 latency, memory, throughput and numerical agreement.	Efficiency Parameters, FLOPs, tokens, energy estimate and performance Pareto front.

Datasets, technology stack and compute

Dataset plan

Dataset	Role	Why it is used
Comma2k19	Prototype and real-data dynamics	33+ hours of road camera, GPS, IMU and CAN signals including speed and steering. Limited route diversity must be reported.
nuScenes / ACT-Bench subset	Diverse urban evaluation	More varied scenes and trajectories; useful for comparison with published action-controllability work.
CARLA	Controlled intervention and closed loop	Repeatable scenes with deliberately varied actions, traffic and weather; supports Python and C++ integration.
Isaac Lab or visual navigation	Small cross-domain transfer	Tests whether representations and action-fidelity metrics transfer beyond cars without creating a second major project.

Python training and research

Python 3.12 PyTorch torchvision / timm OpenCV / PyAV
Hydra MLflow or W&B pytest Ruff

Begin with frozen video/VAE features, then progressively study joint video-action-state pretraining and reusable world-model representations.

C++ and GPU performance

C++20 CMake ONNX Runtime TensorRT CUDA
Triton Nsight GoogleTest

Profile batch-one rollouts and training bottlenecks; introduce TensorRT, custom Triton/CUDA work and low precision only after measurements justify them.

Training and data infrastructure

DDP FSDP / DeepSpeed AMP streaming shards
Docker GitHub Actions DVC optional

Configuration-driven, resumable experiments with data validation, distributed checkpoints, throughput telemetry, fixed splits and Python/C++ equivalence tests.

Recommended development machine

GPU: NVIDIA 24 GB VRAM (RTX 3090/4090, RTX A5000-class)
CPU: 12–16 modern cores
RAM: 64 GB
Storage: 2–4 TB NVMe SSD
OS: Linux preferred; WSL2 acceptable for training

Compute plan: baselines should fit on 12–16 GB VRAM; one 24 GB GPU supports the core research. Budget roughly 8–48 GPU-hours per serious run and hundreds across ablations. Short rented 2–4 GPU runs validate DDP/FSDP scaling. Frontier-scale pretraining is not attempted; scaling claims remain empirical and bounded.

18–24 month timeline

Months 1–2	Specification and data audit Literature map, dataset schemas, synchronisation checks, fixed splits and reproducible data subset.	Gate: trustworthy pipeline
Months 3–4	Deterministic baselines Repeat-state, constant-motion, linear/MLP dynamics and evaluation by time horizon.	Gate: baseline report
Months 5–7	Action-conditioned model Recurrent and small-transformer models; one-step then autoregressive 0.5–2 second rollouts.	Gate: beats baselines
Months 8–10	Action-fidelity benchmark Counterfactual actions, action-agnostic ablation, shuffled inputs and cross-scene evaluation.	Gate: paper-worthy evidence
Months 11–12	Production MVP ONNX/C++20 runtime, correctness tests and p50/p95/p99 profiling; portfolio-ready ML engineering release.	Gate: apply for ML roles
Months 13–15	Uncertainty and distributed training Multiple futures, calibration and OOD tests; DDP/FSDP, sharded data and throughput scaling.	Gate: reliable + scalable
Months 16–18	CARLA closed-loop evaluation Candidate generation, rollout scoring, uncertainty penalty and repeatable driving scenarios.	Gate: closed-loop results
Months 19–21	GPU efficiency and transfer Action-aware token routing, AMP/TensorRT/Triton profiling, plus one compact navigation or manipulation study.	Gate: transferable result
Months 22–24	Publication and release Replication, first-author submission, open-source release, scaling report, project site and PhD proposal.	Gate: submission package

Decision points

Month 4

If data alignment is unreliable, narrow to one source before modelling.

Month 10

If the model ignores actions, prioritise the benchmark and objective design over scaling.

Month 18

If closed-loop results are unstable, publish open-loop findings and retain CARLA as future work.

Two success points: month 12 delivers an employable end-to-end ML system; month 24 delivers a research programme spanning controllability, distributed training, GPU efficiency and cross-domain embodied transfer.

Success criteria, outputs and risks

Research success

- A precise contribution beyond model implementation.
- Action-conditioned gains against an identical action-free baseline.
- Counterfactual evaluation and one cross-domain transfer study.
- Ablations, negative results and statistical uncertainty.
- A first-author paper, workshop submission or public preprint.

Engineering success

- Reproducible training from a documented data subset.
- Tested Python, C++ and GPU-optimised inference paths.
- Measured batch-one latency, memory and multi-GPU scaling.
- Resumable distributed runs, Docker, CI and versioned experiments.
- Short interactive or recorded closed-loop demonstration.

Principal risks and mitigations

Risk	Mitigation
Model ignores the action signal.	Action-free control model, shuffled actions, counterfactual sweeps and explicit action-consistency loss.
Visual generation consumes the compute budget.	Make structured latent prediction primary; keep decoding diagnostic and low resolution.
Observational data cannot validate interventions.	Use CARLA for controlled counterfactual tests and clearly separate real-data from simulator claims.
Scaling becomes performative rather than scientific.	Report throughput, memory and convergence from controlled 1–4 GPU tests; make no fleet-scale claims.
Project expands indefinitely.	Freeze the driving MVP at month 12; keep the second domain to one transfer experiment.
Solo research lacks feedback or references.	Seek periodic academic review; retain project leadership and first authorship while collaborating where valuable.

Release package

- Open repository
- First-author paper
- Experiment tables
- Model/data cards
- Scaling report
- C++/CUDA benchmark
- CARLA demo
- Transfer study

PhD and career direction: position the work as “Efficient and Action-Controllable Multimodal World Models for Embodied Intelligence.” Driving supplies the primary testbed; scalable pretraining, GPU systems and transfer keep the work relevant to Wayve, NVIDIA Physical AI, DeepMind and general multimodal foundation-model teams.

Selected foundations

Wayve, “GAIA-4: A generative world model for closed-loop driving simulation,” 2026 — <https://wayve.ai/thinking/gaia-4/>
NVIDIA, “Cosmos World Foundation Model Platform for Physical AI” — <https://research.nvidia.com/labs/dir/cosmos1/>
NVIDIA GEAR, “Generalist Embodied Agent Research” — <https://research.nvidia.com/labs/gear/>
Arai et al., “ACT-Bench,” ICLR World Models Workshop, 2025 — <https://iclr.cc/virtual/2025/37570>
comma.ai, “Comma2k19” — <https://github.com/commaai/comma2k19> · CARLA — <https://carla.readthedocs.io/>